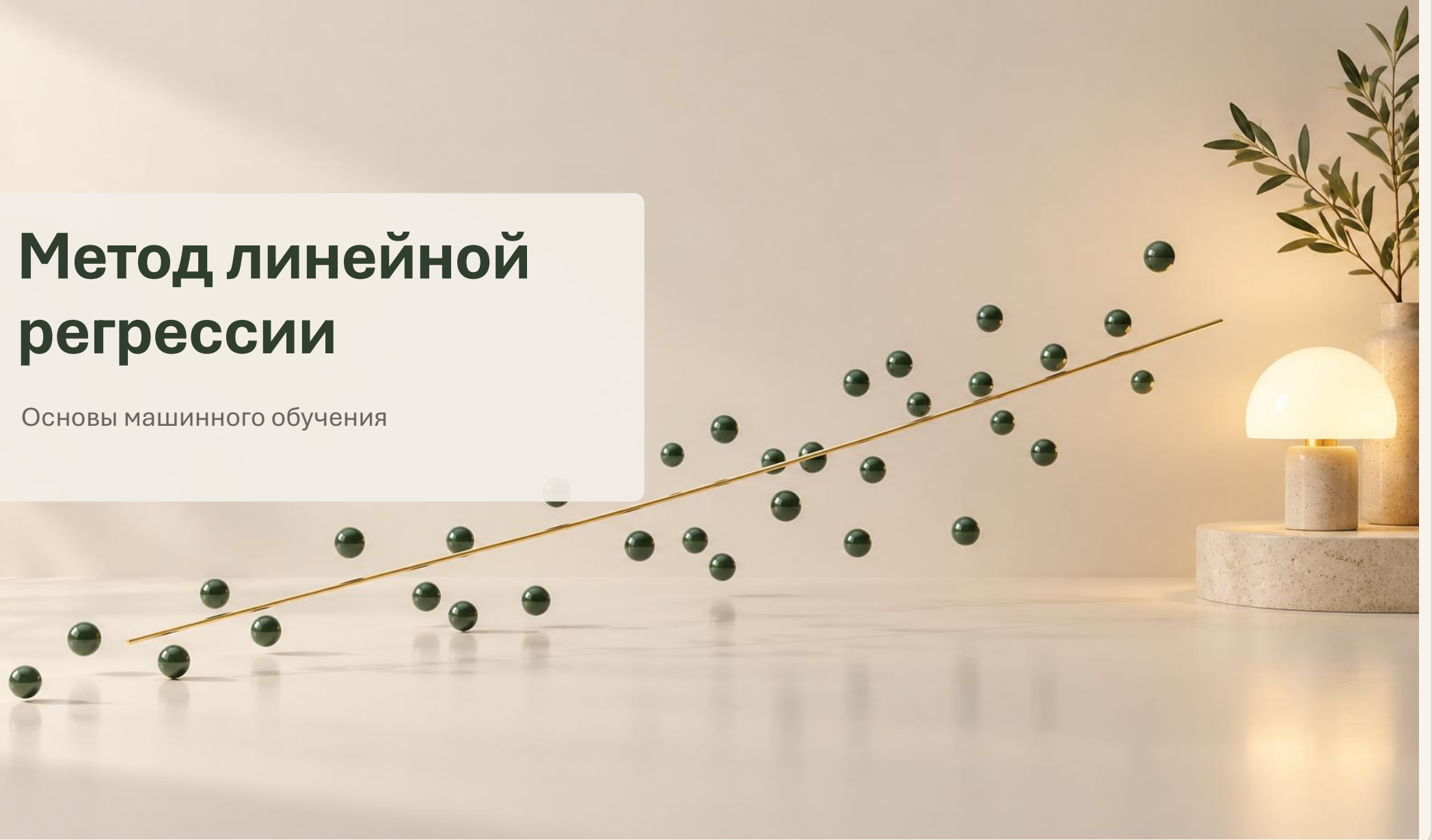


Метод линейной регрессии

Основы машинного обучения



План презентации

1. Постановка задачи и области её применения
2. Математическая основа метода
3. Пошаговый численный пример
4. Пример решения на языке Python
5. Ограничения метода и сравнение с альтернативами





1. Постановка задачи и области её применения

Постановка задачи

Линейная регрессия решает задачу восстановления зависимости между набором признаков объекта и некоторой вещественной целевой переменной.

Один признак

площадь квартиры → стоимость
квартиры

Несколько признаков

(площадь, этаж, расстояние до метро)
→ стоимость



2. Математическая основа метода

Интерполяция, аппроксимация, регрессия

Интерполяция

Функция должна проходить через все заданные точки

$$f(x_i) = y_i$$

Аппроксимация

Точного совпадения не требуется — минимизируется ошибка приближения

$$f(x) \approx g(x)$$

Регрессия

Исходные данные рассматриваются как наблюдения, содержащие некоторую закономерность и шум

Задача — восстановить зависимость между признаками и целевой переменной и использовать её для прогнозирования новых значений

$$y_i = f(x_i) + \varepsilon_i$$

Модель с одним признаком

Самая простая форма линейной регрессии — прямая на плоскости

Для одного признака простейшая линейная модель имеет вид:

$$\hat{y} = w_0 + w_1x$$

где

x — признак;

$\hat{y}$ — предсказание модели;

w_0 — свободный коэффициент;

w_1 — вес признака.

Пример

$$\hat{y} = 2 + 0.1x$$

Если получена такая модель, то при увеличении x на единицу предсказание возрастает на 0.1.

В ML это удобно: коэффициенты модели можно интерпретировать как вклад признаков в прогноз.

Почему модель называется линейной

В общем случае линейная регрессия записывается как линейная комбинация базисных функций:

$$f(x) = \sum_{j=0}^m w_j \varphi_j(x)$$

Линейность здесь относится именно к коэффициентам w_j . Поэтому модель может быть нелинейной по x , но линейной по параметрам

Пример

$$\hat{y} = w_0 + w_1x + w_2x^2$$

Такая модель тоже является линейной по параметрам w_0, w_1, w_2 , хотя её графиком будет парабола.

Выбирая базисные функции, можно менять форму зависимости, сохраняя линейность модели по параметрам.

Метод наименьших квадратов

Пусть имеется обучающая выборка:

$$(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)$$

Для каждого объекта модель получает прогноз:

$$\hat{y}_i = w_0 + w_1 x_i$$

Разница между истинным значением и прогнозом называется остатком:

$$e_i = y_i - \hat{y}_i$$

Сумма квадратов остатков:

$$SSE(w_0, w_1) = \sum_{i=1}^N (y_i - w_0 - w_1 x_i)^2$$

$$(\hat{w}_0, \hat{w}_1) = \operatorname{argmin}_{w_0, w_1} SSE(w_0, w_1)$$

Выбираем такие w_0 и w_1 , при которых SSE минимальна.

Коэффициенты для одного признака

Для простой линейной регрессии коэффициенты можно записать явно.

$$\hat{w}_1 = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^N (x_i - \bar{x})^2}$$

$$\hat{w}_0 = \bar{y} - \hat{w}_1 \bar{x}$$

$$\bar{x} = (1/N) \sum_{i=1}^N x_i$$

$$\bar{y} = (1/N) \sum_{i=1}^N y_i$$

$$\hat{w}_1 = \text{cov}(x,y) / \text{var}(x)$$

Коэффициент наклона определяется отношением ковариации x и y к дисперсии x . Поэтому знак ковариации определяет направление зависимости: при положительной ковариации $\hat{w}_1 > 0$, при отрицательной — $\hat{w}_1 < 0$.

w_1 показывает изменение прогноза при увеличении x на единицу

Несколько признаков

Например, стоимость квартиры может зависеть от площади, числа комнат и расстояния до метро. Тогда модель принимает вид:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

x_1 = площадь
 x_2 = число комнат
 x_3 = расстояние до метро

Каждый коэффициент w_j показывает изменение прогноза при изменении соответствующего признака x_j на единицу при фиксированных остальных признаках

Матричная запись

Матрица X хранит объекты по строкам, признаки — по столбцам.

$$X = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1n} \\ 1 & x_{21} & x_{22} & \cdots & x_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{N1} & x_{N2} & \cdots & x_{Nn} \end{pmatrix}$$

Матрица признаков: строки — объекты, столбцы — признаки

$$w = (w_0, w_1, \dots, w_n)^T$$

Вектор параметров модели

$$y = (y_1, y_2, \dots, y_N)^T$$

Вектор истинных значений целевой переменной

$$\hat{y} = Xw$$

Предсказания модели для всей выборки

$$\hat{w} = \operatorname{argmin}_w \|y - Xw\|_2^2 = \operatorname{argmin}_w \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Задача обучения: найти коэффициенты, минимизирующие сумму квадратов ошибок

$$X^T X \hat{w} = X^T y$$

Нормальные уравнения метода наименьших квадратов

$$\hat{w} = (X^T X)^{-1} X^T y$$

Явное решение при обратимой $X^T X$

От математической модели к ML

В задаче машинного обучения мы уже не вычисляем коэффициенты вручную. Процесс выглядит так:

1. Формируем признаки X и целевую переменную y
2. Делим данные на обучающую и тестовую выборки
3. На обучающей выборке модель подбирает коэффициенты
4. Получаем прогнозы для новых объектов
5. Сравниваем прогнозы с истинными значениями тестовой выборки с помощью метрик

```
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

model.coef_      # веса признаков
model.intercept_ # свободный коэффициент
```



3. Пошаговый численный пример

Пошаговый численный пример

Рассмотрим упрощённую задачу оценки стоимости квартиры по площади. Цена измеряется в миллионах рублей. Ищем модель:

$$\hat{y} = w_0 + w_1x$$

Площадь, м ²	Цена, млн руб.
30	5
40	6
50	8

Средние значения

$$\begin{aligned}\bar{x} &= \\ \frac{30 + 40 + 50}{3} \\ \bar{x} &= 40\end{aligned}$$

$$\begin{aligned}\bar{y} &= \\ \frac{5 + 6 + 8}{3} \\ \bar{y} &= 19/3\end{aligned}$$

Регрессионная прямая с константой проходит через точку $(\bar{x}, \bar{y}) = (40, 19/3)$

Дальше посчитаем, насколько каждое наблюдение отклоняется от средних значений, и используем эти отклонения для поиска наклона прямой

Находим коэффициент w_1

$$\hat{w}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

числитель:

$$(30-40)(5-19/3) + (40-40)(6-19/3) + (50-40)(8-19/3)$$
$$= (-10)(-4/3) + 0 + 10(5/3)$$
$$= 40/3 + 50/3 = 30$$

знаменатель:

$$(30-40)^2 + (40-40)^2 + (50-40)^2$$
$$= 100 + 0 + 100 = 200$$

$$\hat{w}_1 = 30/200 = 0.15$$

В полученной модели увеличение площади на 1 м² увеличивает прогноз цены на 0.15 млн рублей

Модель и прогноз

Находим свободный коэффициент и используем прямую для нового объекта.

$$\begin{aligned}\hat{w}_0 &= \bar{y} - \hat{w}_1 \bar{x} \\ &= 19/3 - 0.15 \cdot 40 \\ &= 19/3 - 6 \\ \hat{w}_0 &= 1/3\end{aligned}$$

$$\hat{y} = 1/3 + 0.15x$$

Полученная модель оценивает цену квартиры по её площади. Теперь можно подставить новое значение x .

$$x = 60 \text{ м}^2$$

$$\begin{aligned}\hat{y} &= 1/3 + 0.15 \cdot 60 \\ &= 1/3 + 9 \\ \hat{y} &\approx 9.33\end{aligned}$$

прогноз: 9.33 млн руб.



6. Пример решения на языке Python

Python №1

Решаем предыдущий пример не вручную, а через LinearRegression.

```
import numpy as np
from sklearn.linear_model import
LinearRegression

X = np.array([
    [30],
    [40],
    [50]
])

y = np.array([5, 6, 8])

model = LinearRegression()
model.fit(X, y)
```

```
print("Свободный коэффициент:",
model.intercept_)
print("Коэффициент площади:", model.coef_[0])

prediction = model.predict([[60]])

print("Цена квартиры 60 м²:", prediction[0])
```

$$w_0 = 0.333, w_1 = 0.15$$

$$\hat{y} = 0.333 + 0.15x$$

$$\hat{y}(60) = 9.333$$

Python №1: визуализация

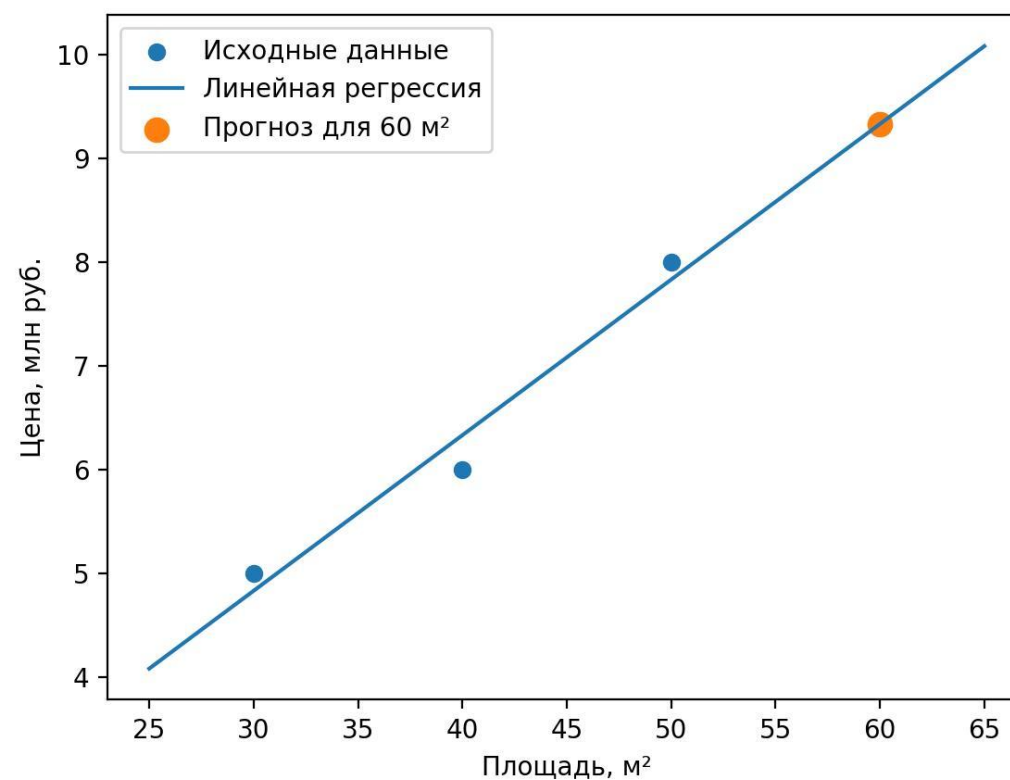
```
import matplotlib.pyplot as plt

x_line = np.linspace(25, 65, 100).reshape(-1, 1)
y_line = model.predict(x_line)

x_new = np.array([[60]])
y_new = model.predict(x_new)

plt.scatter(X[:, 0], y, label="Исходные данные")
plt.plot(x_line[:, 0], y_line, label="Линейная
регрессия")
plt.scatter(
    x_new[:, 0],
    y_new,
    s=80,
    label="Прогноз для 60 м²"
)

plt.xlabel("Площадь, м²")
plt.ylabel("Цена, млн руб.")
plt.legend()
plt.show()
```



Python №2

Компания хочет оценить зависимость продаж от рекламного бюджета

Имеются 12 наблюдений за рекламным бюджетом и продажами. Признак X — рекламный бюджет в тысячах рублей. Целевая переменная y — продажи в штуках.

Здесь появляется важное отличие от первого учебного примера: данные делятся на train и test.

X = рекламный бюджет, тыс. руб.
 y = продажи, шт.

train → обучение коэффициентов

test → проверка на новых объектах

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score

X = np.array([
    [10], [15], [20], [25],
    [30], [35], [40], [45],
    [50], [55], [60], [65]
])

y = np.array([
    94, 108, 119, 137,
    146, 164, 171, 189,
    202, 214, 231, 239
])
```

Python №2: train/test и прогноз

Модель учится только на обучающей части данных

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Коэффициент:", model.coef_[0])
print("Свободный член:", model.intercept_)

print("MAE:", mean_absolute_error(y_test, y_pred))
print("R²:", r2_score(y_test, y_pred))
```

```
new_budget = np.array([[52]])

print(
    "Прогноз продаж:",
    model.predict(new_budget)[0]
)
```

Модель определяет параметры только по X_{train} и y_{train} . После этого её проверяют на объектах, которых она не видела во время обучения.

$$w_1 \approx 2.645$$

$$w_0 \approx 68.366$$

$$\text{MAE} \approx 1.63$$

$$R^2 \approx 0.9986$$

$$\hat{y}(52) \approx 205.9$$

Метрики качества

Mean Absolute Error показывает среднюю абсолютную величину ошибки прогноза. Например, $MAE = 5$ означает, что модель в среднем ошибается примерно на 5 единиц товара.

$$MAE = (1/N) \sum_{i=1}^N |y_i - \hat{y}_i| = \frac{SSE}{N}$$

R^2 сравнивает модель с простым прогнозом, который для всех объектов выдаёт среднее значение $\bar{y}$

$R^2 = 1$ идеальные прогнозы

$R^2 = 0$ не лучше предсказания средним

$R^2 < 0$ хуже предсказания средним

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

На практике линейную регрессию нельзя оценивать только по качеству на обучающей выборке: необходима проверка на данных, которые не использовались при подборе коэффициентов.

Python №3

Требуется спрогнозировать время доставки заказа.
Пусть известны три признака:

x_1 = расстояние до клиента
 x_2 = число товаров
 x_3 = час пик

$x_3 = \{ 1, \text{если час пик} \}$
 $0, \text{если обычное время} \}$

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + w_3x_3$$

В отличие от первого и второго примера, теперь один объект описывается сразу несколькими признаками. Каждому признаку соответствует свой коэффициент, характеризующий изменение прогноза при фиксированных остальных признаках.

Python №3: данные

Используем DataFrame, чтобы признаки были названы понятно.

```
import pandas as pd

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error

data = pd.DataFrame({
    "distance_km": [
        2, 3, 4, 5, 6, 7,
        8, 9, 10, 11, 12, 13
    ],
    "items": [
        1, 4, 2, 6, 3, 5,
        2, 7, 4, 6, 3, 8
    ],
    "rush_hour": [
        0, 0, 1, 0, 1, 0,
        1, 0, 1, 1, 0, 1
    ],
    "delivery_time": [
        22, 29, 40, 39, 48, 47,
        55, 57, 65, 70, 60, 79
    ]
})

X = data[["distance_km",
          "items",
          "rush_hour"]]

y = data["delivery_time"]
```

Python №3: обучение и прогноз

Модель возвращает коэффициенты для каждого признака

```
X_train, X_test, y_train, y_test =  
train_test_split(  
    X,  
    y,  
    test_size=0.25,  
    random_state=42  
)
```

```
model = LinearRegression()  
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Свободный коэффициент:")  
print(model.intercept_)
```

```
print("Коэффициенты признаков:")
```

```
for feature, coef in zip(  
    X.columns,  
    model.coef_  
):  
    print(feature, coef)  
  
print(  
    "MAE:",  
    mean_absolute_error(y_test, y_pred)  
)  
  
new_order = pd.DataFrame({  
    "distance_km": [8],  
    "items": [4],  
    "rush_hour": [1]  
})  
  
prediction = model.predict(new_order)  
  
print("Прогноз времени доставки:",  
    prediction[0])
```

Python №3: обучение и прогноз

$$w_0 \approx 15.30$$

$$w_{\text{distance}} \approx 4.10$$

$$w_{\text{items}} \approx 0.56$$

$$w_{\text{rush}} \approx 6.34$$

$$\text{MAE} \approx 2.79 \text{ мин}$$

Для заказа 8 км, 4 товара, час пик

$$\hat{y} \approx 56.69 \text{ мин}$$

Интерпретация коэффициентов

$$\hat{y} = w_0 + w_1 \cdot \text{distance} + w_2 \cdot \text{items} + w_3 \cdot \text{rush_hour}$$

$$w_1 \approx 4.10 \rightarrow + 1 \text{ км} \approx + 4.10 \text{ мин}$$

$$w_2 \approx 0.56 \rightarrow + 1 \text{ товар} \approx + 0.56 \text{ мин}$$

$$w_3 \approx 6.34 \rightarrow + \text{час пик} \approx + 6.34 \text{ мин}$$

Именно возможность напрямую анализировать веса отличает линейную регрессию от многих более сложных моделей

Что делает `fit()` внутри `LinearRegression`

При вызове `model.fit(X, y)` модель получает матрицу X и целевой вектор y

$$\hat{w} = \arg \min \|y - Xw\|_2^2$$

`fit ()` подбирает коэффициенты, минимизирующие сумму квадратов ошибок

На практике явную обратную матрицу обычно не вычисляют: `LinearRegression` использует численный решатель задачи наименьших квадратов.

$$\hat{w} = (X^T X)^{-1} X^T y$$

Аналитическое решение при обратимой $X^T X$

```
model.fit(X, y)

model.coef_
model.intercept_

model.predict(new_objects)
```



5. Ограничения метода и сравнение с альтернативами

Когда линейная регрессия подходит

- целевая переменная — числовая
- между признаками и ответом есть примерно линейная зависимость
- важна интерпретируемость модели
- нужна простая baseline-модель
- объём данных и сложность задачи не требуют более мощного алгоритма

Линейную регрессию часто используют как baseline: сначала строят простую модель и сравнивают с ней более сложные алгоритмы. Если существенного выигрыша в качестве нет, усложнение модели может быть неоправданным

простая baseline → точка
сравнения для сложных моделей

Ограничения: нелинейность и выбросы

Если реальная зависимость имеет вид

$$y \approx x^2$$

то обычная модель

$$\hat{y} = w_0 + w_1 x$$

не сможет хорошо её описать

В OLS остатки возводятся в квадрат, поэтому наблюдение с большой ошибкой получает непропорционально большой вклад в функцию потерь и может заметно сместить регрессионную прямую

$$(y_i - \hat{y}_i)^2$$

Ограничения: мультиколлинеарность и экстраполяция

Мультиколлинеарность возникает, когда признаки сильно связаны между собой. Например:

$x_1 = \text{площадь в м}^2$

$x_2 = \text{та же площадь в футах}^2$

Предсказания ещё могут быть приемлемыми, но отдельные коэффициенты становятся нестабильными и плохо интерпретируемыми

Если модель обучалась только на квартирах площадью 30–80 м², то прогноз для квартиры площадью 500 м² является экстраполяцией

30–80 м² → обучающий диапазон

500 м² → экстраполяция

Закономерность, найденная внутри обучающего диапазона, не обязана сохраняться далеко за его пределами

Как сравнивать альтернативы

Альтернативы можно разделить на три направления: регуляризация, преобразование признаков и переход к другому классу моделей

Сравнивать линейную регрессию можно как с другими линейными моделями, так и с моделями, которые по-другому описывают зависимость между признаками и целевой переменной

Основной вопрос выбора модели: важнее интерпретируемость и простота или способность описывать сложную нелинейную зависимость?

Linear Regression
→ Ridge / Lasso
→ Polynomial Regression
→ Decision Tree

Ridge-регрессия

Ridge полезна, например, при большом количестве признаков и мультиколлинеарности. Она снижает чувствительность коэффициентов к небольшим изменениям данных

$$L = \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^n w_j^2$$

То есть к обычной ошибке SSE добавляется штраф за слишком большие веса. Чем больше α , тем сильнее регуляризация

обычная регрессия: минимизирует ошибку
Ridge: ошибка + контроль величины коэффициентов

Lasso

В Lasso к функции потерь добавляется сумма модулей коэффициентов. Особенность метода — некоторые веса могут стать равными нулю.

$$L = \sum_{i=1}^N (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^n |w_j|$$

Поэтому Lasso одновременно выполняет регуляризацию и отбор признаков

$w_j = 0 \rightarrow$ признак фактически
исключён

Полиномиальная регрессия и дерево решений

Обе альтернативы помогают при нелинейной зависимости, но делают это по-разному

Полиномиальная регрессия добавляет новые признаки:

$$x^2, x^3, \dots$$

$$\hat{y} = w_0 + w_1x + w_2x^2$$

Модель остаётся линейной по коэффициентам, но может описывать криволинейные зависимости.

Дерево решений вообще не предполагает линейной зависимости. Оно последовательно разбивает пространство признаков на области и в каждой области выдаёт своё предсказание

Плюсы:

- моделирует нелинейность
- автоматически учитывает взаимодействия признаков
- не требует линейной зависимости

Минусы:

- отдельное дерево легко переобучается
- прогноз получается кусочно-постоянным
- нет такой простой интерпретации коэффициентов



Краткое сравнение методов

Метод	Когда использовать	Основная особенность
Linear Regression	Зависимость примерно линейная, важна интерпретация	Простая baseline-модель
Ridge	Много признаков, мультиколлинеарность	L_2 -регуляризация, уменьшает коэффициенты
Lasso	Много признаков, часть из них может быть избыточной	L_1 -регуляризация, может занулять коэффициенты
Polynomial Regression	Зависимость нелинейная, но достаточно гладкая	Добавление степеней признаков
Decision Tree	Сложная нелинейная зависимость	Разбиение пространства признаков

Итог

Линейная регрессия решает задачу прогнозирования числовой величины. В машинном обучении параметры автоматически определяются по обучающей выборке, после чего модель используется для прогнозирования новых значений

$$\hat{y} = w_0 + \sum_{j=1}^n w_j x_j$$

$$\hat{w} = \operatorname{argmin} \|y - Xw\|_2^2$$

Преимущества:

- Простота
- вычислительная эффективность
- интерпретируемость коэффициентов
- удобство в качестве baseline-модели

Недостатки:

- ограниченность линейной зависимостью
- чувствительность к выбросам
- проблемы при сильно коррелирующих признаках
- ненадёжность экстраполяции

Источники

1. Habr: «Основы линейной регрессии» — постановка регрессии, МНК, матричная форма, базисные функции.

<https://habr.com/ru/articles/514818/>

2. Яндекс Практикум: «Линейная регрессия: определение, формула и примеры использования в ML» — задачи ML, применение и baseline-модель.

<https://practicum.yandex.ru/blog/что-такое-lineynaya-regressiya/>

3. scikit-learn: LinearRegression — реализация OLS, coef_, intercept_, predict.

https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html